

myERP — Product Requirements

myERP — Product Requirements Document

Version 1.0 · 22 September 2026 · Author: Tam Tran Purpose: the requirements for myERP, an agent-native finance system of record, written so that a reader who has never used an ERP can follow it, and precise enough for Opsera Forge to generate a specification, an architecture and user stories from it.

0. How to read this document

- Words in **bold** on first use are defined in the glossary (§15). If a sentence still does not make sense, read the glossary entry and come back.
 - Every requirement has an id (`FR-LED-01`) and a release tag. **R1** is what gets built first, on Opsera Forge at SF Enterprise Hackathon 2.0 on 23 September 2026. **R2** is the following 90 days. **R3** is 6 to 12 months out. When a stage in Forge asks "what is in scope", the answer is: everything tagged R1, in the order of §7.
 - Where a requirement quotes an exact number (for example "6 of 70 documents move by USD 13,103.51"), that number comes from a hand-built reference implementation described in Appendix B and is the acceptance test for the feature.
-

1. Vision

One line. myERP is the agent-native finance system of record for usage-based, multi-entity companies: one universal ledger with billing, tax, withholding and audit evidence inside it, audit-ready by construction, that runs either as the company's ERP or as the accounting hub beside the ERP it already has.

In plain words. An **ERP** is the software a company uses to keep its books: what it sold, what it bought, what it owes, what tax it collected, and the proof that all of it is right. The four systems most large companies run (SAP, Oracle, Workday, NetSuite) each solved parts of this well, but they were designed before three things existed: companies that sell by the token, auditors who expect evidence to be reproducible rather than screenshotted, and AI agents that can do finance work if the system can safely let them. myERP takes the best idea from each of those four systems (§5), leaves out the parts that exist for historical reasons, and is designed around those three things from the first table onward.

Tagline: tax-first, audit-first, agent-native, from the ledger up.

2. The problem

2.1 What an ERP does today

1. It records every transaction as accounting entries in a **ledger**, so that the company's balance sheet and income statement can be produced.

2. It manages the flows around those entries: buying (**procure-to-pay**), selling (**order-to-cash**), paying people, and closing the books each month (**period close**).
3. It decides how much tax belongs on each transaction, often by calling a separate tax engine.
4. It controls who may do what: who can approve an invoice, who can post a journal, who can change a tax rate.
5. It produces reports and, at audit time, evidence that the controls operated.

2.2 Why it hurts

Pain	What it looks like	Who feels it
Money crosses five products. A usage-based sale goes through a billing system, a tax engine, the ERP, a data warehouse and a tax return. Every hop is a reconciliation in a spreadsheet.	"Why does the warehouse say USD 232,587 of New York tax and the return say USD 231,900?" takes a day to answer.	Controller, tax lead, finance systems
Rules change in place. A tax rate, a posting rule or an approval threshold is edited on a setup screen and is live at once. There is no branch, no diff, no way to ask what the change would have done to last month's invoices.	A rate change goes live with a typo; the error surfaces at the return or in a customer complaint.	Tax lead, auditor
Evidence is screenshots. Proof that a control operated is a picture of a screen, a ticket and a transport log. Nothing can be re-run.	Audit preparation takes weeks; SOX readiness is a project rather than a property.	Controller, auditor
AI is bolted on. A chat window sits on top of a system that cannot say what the bot did, cannot stop it from approving its own work, and cannot prove either.	Finance teams are told to use AI and told not to trust it, in the same meeting.	Everyone
Change is slow. New products, new countries and new tax rules take months because configuration, code and process are tangled.	The company goes multi-entity and the finance stack breaks the week the second registration appears.	Founders, finance systems

2.3 What "better" means here

A controller can sign off on a number because the system shows how it was produced and can produce it again. A tax lead can change a rule and see, before approving, exactly what it moves. An AI agent can do the preparatory work and is physically unable to do the approving. An auditor can re-perform any reconciliation with one action. And none of this requires replacing the ERP the company already runs.

3. Who it is for

Primary customer: companies whose revenue is metered (AI APIs, infrastructure, developer platforms, marketplaces), with 2 to 20 legal entities, selling into 20 or more tax jurisdictions, preparing for or living under SOX 404. **Buyer:** the controller or VP Finance; **champions:** the tax lead and the finance-systems lead.

Persona	Seed user	What they need from myERP
Controller	Daniel Okafor	A trial balance that ties to the warehouse, a close that runs itself, approvals that respect segregation of duties, evidence that is ready when the auditor asks
Tax reviewer / tax lead	Priya Raman	Every tax decision explainable line by line; rule changes reviewed with their money impact before they go live; registrations and filings under control
Tax technology engineer	Tam Tran	Configuration as code, an impact check before merging, an API and an MCP server, a warehouse that re-derives the numbers
Accounts payable specialist	Maya Chen	Invoices captured and matched automatically, holds explained, use-tax accrued without a spreadsheet
Revenue accountant	Sam Lee	Usage events rated into invoices, tax committed at finalization, credits and withholding handled as documents
AI finance assistant	Claude	Governed tools to read, check and propose; a clear refusal when it tries to approve; a record of everything it did
Auditor (external)	—	Re-performable evidence runs, control totals, an audit log that cannot be edited, a ref log of every configuration change
System	myERP	Number ranges, scheduled runs, the merge that takes a change into force

There is no login in R1 and R2 demos: a persona switcher ("Acting as") selects the principal, and API calls carry it in an `X-Actor` header. Authentication is an R3 item (§13).

4. Product principles

- One journal, one truth.** Every entity, every book and every currency post into one wide table of journal lines. There is no subledger-to-ledger reconciliation because there is only one table.
- The books cannot go out of balance.** Debits equal credits is a database constraint, not an application check. Money is stored as integer minor units (cents, yen). Posted entries are never edited; corrections are reversals.
- Every transaction is a document.** It has a type, a gap-free number, a status, lines, links to the documents before and after it, and the accounting it produced.
- Rules are configuration, and configuration is version controlled.** Tax rates, taxability, posting rules, approval processes and segregation-of-duties rules are effective-dated content, changed by branch, diff, check, review and merge, never edited in place.
- Every batch job is an evidence run.** It stores its control totals, its tie-outs and a hashed manifest chained to the previous run, and it can be re-performed.
- Tools compute, models narrate.** Calculations, postings, reconciliations and determinations are deterministic code. The AI model reads, explains, extracts and proposes through typed tools; it never computes a balance or writes SQL.
- Agents are principals with roles and tested limits.** An AI agent is a user with a role, like a person. The rules that stop it approving or posting live in code and schema beneath the model, and every refused attempt is kept as evidence that the control operated.
- Nothing is shown, spoken or filed until it is verified.** Every figure an agent states is matched against the tool results it saw before the answer is released.

9. **Beside the ERP or as the ERP, same code.** In hub mode myERP owns billing, tax, withholding and evidence and posts summarised journals with control totals into Workday, Oracle, SAP or NetSuite; in system-of-record mode its journal is the ledger.
10. **The warehouse re-derives; it does not trust.** A dbt project rebuilds every headline number from the open schema and ties it out against the application before anything is published.
-

5. What we borrow from the best, and what none of them has

Each of the four large systems has ideas worth keeping. myERP takes them by name.

From	The idea	What it means in plain words	Where it lands in myERP
SAP	Universal journal	One line table holds financial, management and tax views instead of several ledgers that must be reconciled	§8.1 the ledger: one <code>journal_lines</code> table with base and extension books , three currencies, quantities
SAP	Document principle, number ranges, document flow	Every transaction is a numbered document with a visible chain (order → receipt → invoice → payment)	§8.2 documents and links
SAP	Condition technique	To decide a tax or an account, look up an ordered list of increasingly specific rules and record which one matched	§8.4 one rule engine for accounting and tax, with the match level stored on every line
SAP	Open-item clearing, residual items	Receivables and payables are open items cleared by payments; a short payment becomes a residual item with a reason	§8.3 clearing; withholding as a residual reason
SAP	Change documents	Every configuration change is recorded field by field	§8.5 change documents, and version control on top
Oracle	Subledger accounting (events → rule sets → journal, draft and final, drill-down)	Accounting is generated from events by declared rules, can be previewed, and every line explains itself	§8.1 accounting events and rule sets
Oracle	Cross-validation rules	Not every combination of account and dimensions is allowed; the system says which are	§8.1 account combinations
Oracle	Party model (party, account, site, tax profile)	One customer or supplier can have many sites, each with its own tax facts	§8.3 parties and registrations
Oracle	Accounting Hub	Post summarised accounting from external systems into the ledger with drill-back	§8.10 coexistence (hub) mode

From	The idea	What it means in plain words	Where it lands in myERP
Oracle	Tax configuration hierarchy (regime → jurisdiction → rate → rules)	Tax content is structured, dated and rule-driven	§8.4 tax configuration
Workday	Effective dating on everything	Every setting has a "from" and "to" date, so history and future changes coexist	§8.5 effective-dated configuration
Workday	Worktags	Dimensions are composable tags with required-tag rules rather than a fixed coding block	§8.1 typed dimensions plus free tags
Workday	Business process framework	Approvals are data (steps, conditions, roles), not code	§8.6 process definitions
Workday	Agent System of Record	Agents are registered with identity, skills and audit like employees	§6 agents as principals
NetSuite	Usage rating inside the suite	Usage records are rated into invoices without leaving the system	§8.7 usage billing
NetSuite	SuiteFlow approvals	Approval routing configured, not coded	§8.6
NetSuite	System Notes	A history no user can edit	§8.9 audit log, made tamper-evident with a hash chain
NetSuite	SuiteCloud Development Framework	Configuration moved as code between accounts	§8.5, extended to branch, diff, check and merge

What none of the four has, and myERP does (each statement is supportable today with the reference implementation in Appendix B; incumbents' configuration tooling — SAP transports, Oracle setup packages, Workday Object Transporter, NetSuite SDF — moves configuration between environments with no diff, no impact analysis and no per-change review):

1. **The impact check replays the books before a rule change is approved.** A proposed change is run against every real document under both the current and the proposed configuration, and the reviewer approves a money number, not a setup screen.
2. **Evidence by construction.** Reconciliations store control totals, named tie-outs and hashed manifests chained run to run, and can be re-performed in one action. We found no incumbent describing re-performable, hash-manifested reconciliation evidence.
3. **Agent limits you can test.** Segregation-of-duties rules that stop an AI agent from approving live beneath the model, in open code, and every refused attempt is stored as evidence. Incumbents govern agents through roles and registries; the difference is a refusal you can test and a record that it happened.
4. **Withholding on both sides of the ledger.** A customer's short payment becomes a withholding receivable that a certificate reclassifies to creditable foreign tax, next to outbound withholding decisions and information returns.
5. **Usage event to journal line in one open schema.** The usage event, the rating, the invoice line, the tax record, the withholding and the journal line are linked rows in one Postgres database that a warehouse

can extract in full.

What the incumbents do better, said plainly: breadth (payroll, procurement, supply chain), country content (tax and e-invoicing formats for 70 to 100+ countries), independent assurance (SOC 1 and 2, ISO, FedRAMP), and ecosystem. myERP ships demonstration tax content, has no attestation, and is designed to sit beside them for exactly that reason.

6. Agent-native design

6.1 The rule: an agent is a principal

Every actor in myERP is a **principal** with a kind (`human` , `agent` , `system`), a display name, roles and an entity scope. Claude is the principal `claude` , kind `agent` , role `agent:assistant` . Every process step names which kinds and roles may perform it. The segregation-of-duties rule **SOD-02, "Agents propose, humans decide"**, denies `approve` and `post` to any principal of kind `agent` , and it is evaluated first on every attempt. This single rule is what makes it safe to give the agent real tools: the worst it can do is propose.

6.2 What the agent does (capabilities)

Capability	What happens	Release
Propose a configuration change in plain English	"Raise the New York City rate to 5% from 1 October" becomes a structured edit, validated against the configuration, then a branch, a commit and a change request opened <i>as Claude</i> through the same API a person uses. Checks run. Claude cannot merge it.	R1
Explain a change request	Claude writes a short review note from the diff and the impact result. Every number in the note is checked against the impact result before the note is shown; an ungrounded number triggers one regeneration and then a warning badge.	R1
Try to approve, and be refused	A visible "Ask Claude to merge" action calls the real merge endpoint as <code>claude</code> . SOD-02 refuses it, the refusal is stored as a blocked review and an audit row. This is the demo climax and a control test.	R1
Ask	Questions answered from governed read tools (trial balance, journal search, line lineage, document detail, tax summary, impact). Every figure in the answer is matched to a tool result before it is released; claims cite records.	R1 stretch, R2 full
Invoice intake	Claude reads a supplier invoice PDF into fields with verbatim quotes; code verifies each quote against the PDF text, checks the arithmetic, scans for prompt injection, matches the supplier, and captures the invoice as Claude. Claude cannot approve it.	R2
Close agent	Claude works the close checklist for an entity and period: runs the integrity and tax evidence runs, reviews holds, withholding, receivables and tie-outs, drafts proposals and a memo.	R2
Proposals	Journal proposals are real manual journals in <code>pending_approval</code> ; task proposals route to a role. Approval is a person's step.	R2

Capability	What happens	Release
Voice	Speech in, the same governed session, verification, then speech out. Nothing is spoken before it is verified.	R2
Continuous monitoring	Scheduled agent runs watch for unbalanced entries, stale holds, nexus thresholds approaching, filing deadlines, and open proposals; they raise tasks, never act.	R2
Contract intelligence	Commercial and tax terms (invoicing entity, gross-up, merchant of record, tax-exclusive pricing) extracted with citations into structured terms that drive billing and withholding.	R2
Parallel-run triage	When two tax engines or two configurations are compared, Claude classifies the variances by root cause and drafts the variance memo with citations.	R2
MCP server	The same governed tools exposed over the Model Context Protocol so Claude Code, Claude Desktop or another agent can use myERP with the same limits.	R2
Auditor agent	A read-only agent for external auditors that re-performs evidence runs and answers PBC requests from run ids.	R3
Golden evals as controls	A fixed set of questions with expected figures computed by SQL at eval time; grading is deterministic (figures, citations, tools used, protected state unchanged, verifier passed). Run before every release and after every model change.	R2

6.3 Guardrails (apply to every capability)

1. **Propose, never approve or post.** Enforced by SOD-02 in the process layer, not by the prompt.
2. **Typed tools only.** Every tool has a strict input schema; there is no raw SQL tool; each call runs in its own transaction and is written to an append-only step table chained into the audit log.
3. **Verified before shown.** A deterministic grounding check matches every figure in an answer to a tool result at the precision written, or to a sum, difference or percentage of two results.
4. **Model settings are configuration.** Model id, effort and caching are settings, so a model upgrade cannot change an accounting result; it can only change which tools are called and how the answer is worded, and the evals measure that.
5. **Refusals are evidence.** A blocked attempt is stored with the rule id and message, and shown in the UI, because a control that operated is what an auditor wants to see.
6. **Injection is assumed.** Text from documents and external systems is data; extraction is structured and quotes are verified against the source.

6.4 How the model is called

Claude through the official Anthropic SDK (`claude-opus-5`, adaptive thinking, prompt caching on the stable system prompt and tool list). Extraction and proposals use structured outputs so the response is schema-valid JSON, never free text parsed by hand. Read tools use tool use with strict schemas. The API key is an environment variable; without it the agent actions explain what is missing and everything else works.

7. Scope by release

7.1 R1 — built on Opsera Forge, 23 September 2026

A single deployable web app (Next.js, TypeScript, Prisma, PostgreSQL on Neon, Tailwind, Vercel) seeded from the files in Appendix A. It shows the ledger, the documents and the tax results of the fictional company Halide Labs on the business date 16 September 2026, and it implements change control for tax configuration end to end with an agent as a governed participant.

Screens, in build order:

1. Persona switcher and X-Actor plumbing
2. **Overview**: tax by jurisdiction, journal and document counts, integrity strip (entries balanced per currency: yes/no)
3. **Universal journal**: search lines by entity, book, account, period; open a line to see its entry, its source document and the tax line it came from
4. **Trial balance** by entity and book, per currency, up to a period
5. **Documents**: list and detail with lines, tax lines, journal entries
6. **Rates** (configuration in force), **Branch editor**, **Change requests** list and detail (diff, checks, impact, reviews), **History** (ref log, commits, tags)
7. **Agent panel** on a change request: propose in plain English, explain, ask Claude to merge
8. **Audit log** with hash chain and a verify action
9. Stretch: **Ask** with read tools; **merge queue** replay of two open change requests together

Definition of done for R1: the acceptance tests in §12 pass at the hosted URL, and the demo path runs in under three minutes: propose the New York City change → diff → run checks → impact 6 of 70 documents move by USD 13,103.51. → Claude explains it → Claude tries to merge and is refused by SOD-02 → Tam is refused by SOD-01 → Priya approves and merges → main moves and the ref log shows who and why → the overview's tax-by-jurisdiction figure for New York City changes when viewed as of the new configuration.

7.2 R2 — the following 90 days

Procure-to-pay with three-way match, holds and payment runs; usage billing with idempotent events, rating, credits and immutable invoices; tax determination behind one engine interface with the stub engine, Stripe Tax and Avalara adapters; withholding both ways; evidence runs with re-performance and export; the dbt warehouse; the full agent layer (§6.2 R2 rows) with the MCP server and golden evals; the first coexistence connector (Workday journal load).

7.3 R3 — 6 to 12 months

Authentication and SSO; continuous close; the auditor portal; jurisdiction packs as code; e-invoicing through an access-point partner; parallel-run studio as a product; bitemporal as-of reporting; the compute-cost allocation engine; SOC 2 Type II.

8. Functional requirements

Tags: **R1** built on Forge tomorrow · **R2** · **R3**. "Must" is binding for the tagged release.

8.1 Ledger (LED)

Id	Requirement	Release
FR-LED-01	The system must hold one universal journal: a <code>journal_lines</code> table where every line carries entity, book (BASE, or an extension book such as TAX that holds only deltas), period, posting date, account, typed dimensions (customer, supplier, product, cost center, department, project, region, channel, trading partner), amounts in transaction, entity and group currency, optional quantity and unit, tax jurisdiction and tax line reference, open-item flag, source document, the rule that produced it and the level at which it matched.	R1 (read from seed), R2 (posting)
FR-LED-02	Every journal entry must balance in every currency column. In R1 the seed is verified on load (sum of <code>amount_minor</code> per currency equals zero) and the overview shows the result; in R2 the rule is a deferred database constraint that rejects the commit.	R1 / R2
FR-LED-03	Money must be integer minor units with a currency code; display as <code>USD 13,103.51</code> , <code>JPY 3,764,480</code> , <code>EUR 65,532.98</code> . Rounding is half-up per line; remainders are posted, never dropped.	R1
FR-LED-04	A trial balance must be available by entity, book and currency up to a period end, with drill-down from any balance to its lines.	R1
FR-LED-05	Any journal line must open its lineage: entry, source document, the tax line it came from, the accounting rule and match level, and (R2) the accounting event with a verified payload hash.	R1 (partial), R2
FR-LED-06	Posted entries are append-only; corrections are reversal entries that reference the original. Locked periods reject postings.	R2
FR-LED-07	Accounting is generated from accounting events by effective-dated rule sets with draft and final modes; every line records the rule set version.	R2
FR-LED-08	Account combinations (account plus dimensions) are validated by cross-validation rules before posting.	R2

8.2 Documents (DOC)

Id	Requirement	Release
FR-DOC-01	Every transaction is a document with a type (customer invoice, supplier invoice, purchase order, receipt, settlement, manual journal, credit purchase, payment run, ...), a gap-free number from a per-entity number range, a status, a party, a currency, dates, header totals, lines, and links to predecessor and successor documents.	R1 (read), R2 (create)
FR-DOC-02	A document's detail must show its lines, its tax lines (jurisdiction, treatment, taxable base, rate, tax, accrual flag), its journal entries and, in R2, its process history including blocked attempts.	R1
FR-DOC-03	Document flow (order → receipt → invoice → payment) is navigable from any document.	R2
FR-DOC-04	Every document stores the configuration version it was computed with, so any result can be re-performed.	R1 (commit hash on tax lines), R2

8.3 Parties, payables, receivables (PTY)

Id	Requirement	Release
FR-PTY-01	One party record with roles (customer, supplier, intercompany), sites with addresses, and a tax profile per site or registration; counterparty VAT/GST numbers are validated and the result stored.	R1 (read), R2
FR-PTY-02	Supplier invoices go through capture → match (two- or three-way) → determine tax → assurance checks (duplicate, tax variance, tax forms, registration) → approve (if required) → post, with holds that explain themselves.	R2
FR-PTY-03	Control accounts are open items cleared by settlements; a short payment becomes a residual item with a reason code (withholding, dispute, discount).	R2
FR-PTY-04	Payment runs have a proposal step, apply withholding, and produce information-return rows.	R2

8.4 Tax determination (TAX)

Id	Requirement	Release
FR-TAX-01	Tax content is effective-dated configuration: regimes, jurisdictions with stacked rate components, taxability by product class, determination rules, recovery, product tax codes. A jurisdiction's combined rate on a date is the sum of components in force on that date.	R1 (rates), R2 (all scopes)
FR-TAX-02	Every tax result is a record with lines: jurisdiction, treatment (<code>taxable</code> , <code>use_tax_accrual</code> , <code>not_taxable</code> , <code>reverse_charge</code> , ...), taxable base, rate, tax, accrual flag, the rule and match level. <code>use_tax_accrual</code> means the supplier charged no tax but the purchase is taxable, so the company accrues the tax itself; it moves the accrued total, not the invoice.	R1 (read), R2
FR-TAX-03	Determination runs behind one engine interface (quote, commit, void, reverse) so engines can be swapped and compared; the stub engine is the default.	R2
FR-TAX-04	Registrations (which entity is registered where) are data, and a determination that starts charging tax where the entity has no registration is a failure, not a warning.	R1 (in the impact check), R2
FR-TAX-05	A four-way reconciliation ties documents = engine = ledger = return draft per period and is an evidence run.	R2

8.5 Configuration and change control (CFG) — the R1 core

Id	Requirement	Release
FR-CFG-01	Configuration is stored as content-addressed commits : a commit is a complete configuration (the full rate table in R1), its parent(s), an author, a message and a SHA-256 hash of its canonical JSON. A commit is never a patch; any commit can be loaded and run on its own.	R1
FR-CFG-02	Refs are the only mutable thing: a branch moves, a tag never moves, and <code>main</code> (the configuration in force) is protected: no direct commits, changes arrive only by merge.	R1
FR-CFG-03	Every move of every ref is appended to a permanent ref log with from-hash, to-hash, actor and reason. Rows are never updated or deleted.	R1

Id	Requirement	Release
FR-CFG-04	A user proposes a change by creating a branch from <code>main</code> , editing values in a branch editor, and committing with a message; each edit addresses a record by its natural key (jurisdiction code, component name, effective-from date), never by position.	R1
FR-CFG-05	A change request has a number (CR-0001...), title, body, source and target refs, head and base hashes, merge-base hash, an effective-from date, status (open, merged, closed), proposer and timestamps.	R1
FR-CFG-06	The change request shows a field-level diff from merge base to head, one row per changed path, for example <code>rates.US-NY-NYC.New York City.rate · 0.045 → 0.05</code> .	R1
FR-CFG-07	Three required checks run on the current head: <code>merge-clean</code> (three-way structural merge at the path level; both sides changing one path differently is a conflict), <code>config-loads</code> (every rate parses in [0, 1), effective dates ordered, every jurisdiction referenced by a tax line exists), and <code>impact</code> (FR-CFG-08). Each run is stored with conclusion, summary and detail.	R1
FR-CFG-08	The impact check replays every document's tax lines through the configuration in force and the proposed one (rule in §10.3) and reports: documents replayed and changed; totals by currency; a by-jurisdiction table with before, after, delta and a new-obligation flag; the top movers; duration; and a one-sentence headline. It fails only when a new obligation appears in a jurisdiction where the document's entity has no registration.	R1
FR-CFG-09	Reviews are append-only and bound to the head hash reviewed: approved, changes requested, or blocked (with the rule id and message). A new commit leaves earlier approvals in place, marked stale.	R1
FR-CFG-10	Merge requires all three checks to have succeeded on the current head, at least one approval on the current head, and an actor the rules allow. It creates a merge commit on <code>main</code> with two parents, moves <code>main</code> , appends the ref log, marks the request merged, and writes an audit row.	R1
FR-CFG-11	A tag can be created on any commit (for example <code>filed-2026-08</code>) and refuses to move afterwards.	R1
FR-CFG-12	Configuration scopes beyond rates (taxability, rules, recovery, accounting rules, process definitions, withholding, billing) are versioned the same way.	R2
FR-CFG-13	A merge queue replays the books with two or more open change requests applied together and reports when each is clean alone and wrong together.	R1 stretch, R2

8.6 Processes and segregation of duties (PRC)

Id	Requirement	Release
FR-PRC-01	Process definitions are configuration: ordered steps, each with a kind (capture, validate, assure, approve, post), the kinds of actor allowed (human, agent, system) and the roles allowed.	R1 (config_change), R2 (all)

Id	Requirement	Release
FR-PRC-02	Segregation-of-duties rules are evaluated in order on every attempt; the first rule that denies is recorded and shown with its message. R1 rules, verbatim: SOD-02 deny when <code>actor.kind == 'agent'</code> ("{name} is an AI agent. Agents may capture, match, determine tax and run assurance checks; approving and posting belong to a person or to the system after a person approves."); SOD-01 deny when <code>actor.id == change_request.opened_by</code> ("{name} prepared {number}. Segregation of duties requires a different approver."); SOD-03 deny when the change touches tax configuration and <code>tax_reviewer</code> is not in the actor's roles ("{number} changes tax configuration. Approval needs the <code>tax_reviewer</code> role, which {name} does not have."). R2 adds SOD-04 (manual postings to restricted accounts need the controller).	R1
FR-PRC-03	A blocked attempt is stored (as a blocked review in R1; as a process step with outcome <code>blocked</code> in R2) and appears in the UI and the audit log.	R1
FR-PRC-04	Approve and merge buttons are shown to every principal and refused by the rules; the refusal, not a hidden button, is the control.	R1

8.7 Billing and receivables (BIL)

Id	Requirement	Release
FR-BIL-01	Usage events are ingested idempotently (a retry storm drops duplicates), aggregated, rated against price books and contracts, with prepaid credits applied in grant order.	R2
FR-BIL-02	Invoices carry advisory tax while draft and committed tax at finalization; finalized invoices are immutable; corrections are credit notes.	R2
FR-BIL-03	Receipts clear open items; a short payment that is withholding creates a withholding receivable and a certificate reclassifies it.	R2

8.8 Withholding (WHT)

Id	Requirement	Release
FR-WHT-01	Withholding is a first-class object on both sides: outbound decisions on supplier payments (treaty rates, certificates on file, information-return previews) and inbound short-pays from customers.	R2

8.9 Evidence and audit (EVD)

Id	Requirement	Release
FR-EVD-01	An audit log records every action with actor, action, object, summary and payload; each row stores the previous row's hash and its own hash, and a verify action recomputes the chain. Rows are never updated or deleted.	R1
FR-EVD-02	An evidence pack for a change request (diff, checks, reviews, ref-log rows) can be exported as one JSON with a SHA-256 over the canonical body.	R1 stretch, R2

Id	Requirement	Release
FR-EVD-03	Every batch job (reconciliation, extract, integrity check, warehouse build) is an evidence run with control totals, named tie-outs (left, right, delta), artifacts with hashes and a manifest chained to the previous run; a run can be re-performed and exported as a zip.	R2
FR-EVD-04	A controls catalogue maps evidence runs to control statements; an auditor portal serves PBC requests from run ids.	R3

8.10 Reporting, warehouse, integration (RPT, API, HUB)

Id	Requirement	Release
FR-RPT-01	An overview shows tax by jurisdiction (table and bars), filterable by entity and month, with a toggle to view the same totals under an open change request's proposed configuration.	R1
FR-RPT-02	Journal search, trial balance and document search are live queries, not batch reports.	R1
FR-RPT-03	A dbt warehouse re-derives every headline number from a full extract with control totals; tests are controls; nothing is published unless every control passes; a semantic layer exposes governed metrics.	R2
FR-API-01	Every screen is backed by a JSON API under <code>/api</code> ; every request carries the acting principal in <code>X-Actor</code> ; a refused action returns 403 with the rule id and message.	R1
FR-API-02	The agent endpoints (<code>POST /api/change-requests</code> with <code>X-Actor: claude</code> , checks, merge) let an external agent participate under the same rules.	R1
FR-API-03	An MCP server exposes the governed tool registry; an outbox of change events and <code>updated_since</code> extraction feed a warehouse.	R2
FR-HUB-01	In hub mode, final journal lines are summarised per (book, external account mapping, dimensions, period) into external journal batches with control totals, exported in the target ERP's format (Workday EIB, Oracle FBDI, SAP journal upload, NetSuite CSV), recorded as evidence runs; a reconcile job ties the external trial balance back.	R2 (Workday), R3

9. Data model

9.1 R1 tables (seeded from Appendix A)

Table	Plain meaning	Key fields
<code>entities</code>	The company's legal entities (4)	id, name, country, entity currency, group currency
<code>accounts</code>	The chart of accounts (51)	code, name, type, normal balance, control type
<code>principals</code>	People, the agent and the system (7)	id, kind, display name, title, roles[]
<code>parties</code>	Customers and suppliers (35)	id, name, type, roles, country

Table	Plain meaning	Key fields
documents	Transactions (70 with tax; 151 in the full seed)	number, type, entity, party, currency, dates, status, taxable, tax, accrued
tax_lines	The tax result per document line per jurisdiction (608)	document, line, jurisdiction, treatment, taxable base, rate, tax, accrual
journal_entries	Accounting documents (133)	number, entity, book, type, dates, currency, source document, rule set
journal_lines	The universal journal (1,147)	entry, line, entity, book, account, amounts in three currencies, quantity, dimensions, tax jurisdiction, source document, rule, match level
registrations	Where each entity is registered for tax (24)	owner, regime, jurisdiction, number, status
jurisdictions, rate_components	The rate table (15 jurisdictions, 27 components)	code, name, regime; level, name, rate, effective from/to
config_commits, refs, ref_log	Version control	hash, parent, tree JSON, author, message; name, kind, commit, protected; from, to, actor, reason
change_requests, checks, reviews	Change control	see FR-CFG-05, -07, -09
audit_log	Tamper-evident history	actor, action, object, summary, prev hash, hash

9.2 R2 additions

Accounting events, account combinations and cross-validation rules, number ranges, document lines and distributions, document links, open items and clearings, holds, process instances and steps, usage events and aggregates, price books, contracts, credit grants and credit ledger, tax records, withholding events, tax forms and certificates, evidence runs, control totals, tie-outs, artifacts, agent sessions, steps and proposals, and the full set of configuration scopes.

10. Business rules (exact)

10.1 Ledger

- Sum of `amount_minor` per (entry, currency) = 0; sum of entity-currency and group-currency amounts per entry = 0.
- Amounts are integers in minor units; JPY has 0 decimals; conversion to minor units rounds half-up.
- Posted lines are never updated or deleted (R2: enforced by trigger and revoked privileges).

10.2 Combined rate

$\text{combined}(\text{jurisdiction}, \text{date}, \text{config}) = \sum \text{component.rate}$ where $\text{effective_from} \leq \text{date}$ and $(\text{effective_to} \text{ is null or } \text{date} \leq \text{effective_to})$. New York City on 16 Sep 2026 = 0.04 + 0.045 + 0.00375 =

0.08875.

10.3 Impact replay

For every document and each of its tax lines:

- if `treatment ∈ {taxable, use_tax_accrual}`: `new_minor = round_half_up(taxable_base_minor × combined(line.jurisdiction, document.date, head))`; otherwise `new_minor = 0`
- `old_minor` is the same function under the base configuration and must equal the stored `tax_minor` (the first alignment test)
- `accrual = false` contributes to the document's tax delta; `accrual = true` to its accrued delta; a document changed if either `delta ≠ 0`
- outputs: replayed, changed, totals by currency, by-jurisdiction rows sorted by `|delta|` with `new_obligation = (base == 0 and head ≠ 0)`, top 25 movers, duration, headline "{changed} of {replayed} documents move by {delta by currency}." or "No change: {replayed} documents replayed, none moved."
- the check fails only when a `new_obligation` jurisdiction has no registration row for the document's entity

10.4 Change control

- `main` is protected; only merges move it. Tags never move. Ref-log, review, check and audit rows are append-only.
- Mergeability = three successful checks on the current head + one approval on the current head + an allowed actor.
- Rules run in the order SOD-02, SOD-01, SOD-03; the first denial is the one stored and shown.
- An approval is bound to the head hash it reviewed; a new commit marks it stale.

10.5 Money display

Currency code, space, thousands separators, two decimals except JPY: `USD 13,103.51`, `JPY 3,764,480`.

11. Non-functional requirements

Area	Requirement
Performance	The impact replay over the seed (70 documents, 608 lines) completes in under one second in the browser-visible path; the design scopes replay by entity and month so 100,000 documents stay interactive. Screens load in under two seconds on the seed.
Determinism	Replay, checks, diffs and hashes are pure functions with unit tests; the same input always gives the same output and the same hash.
Auditability	Every write goes through the API with an <code>X-Actor</code> ; every refused action is stored; the audit chain verifies.
Security posture	No authentication in R1 and R2 demos (persona switcher); secrets in environment variables; no raw SQL from the model; document text treated as untrusted input. Authentication, SSO and entity-scoped authorization in R3.

Area	Requirement
Testability	Unit tests load <code>answer_key.json</code> and assert to the cent; SOD scenarios are tests; the seed loads from files in one command.
Deployment	One deployable (Next.js on Vercel) with a managed PostgreSQL (Neon); seed and migrations run from the repo; the hosted URL is the submission.
Accessibility and UI	Keyboard-navigable tables, readable at phone width, money right-aligned, dates ISO, no colour as the only signal for pass/fail.

12. Acceptance tests for R1

#	Scenario	Expected
L1	Seed loads	4 entities, 51 accounts, 7 principals, 35 parties, 70 documents, 608 tax lines, 133 journal entries, 1,147 journal lines, 24 registrations, 15 jurisdictions, 27 components; <code>main</code> points at the initial commit; ref log has one row
L2	Integrity	Sum of <code>amount_minor</code> per currency across all journal lines = 0 for USD, EUR, GBP, JPY, SGD; the overview shows "balanced"
L3	Lineage	Opening any journal line with a source document shows the document; a line with a tax jurisdiction shows its tax line
A	Replay with no edit	No change: 70 documents replayed, none moved.; every line's <code>old_minor</code> equals its stored <code>tax_minor</code>
B	New York City 0.045 → 0.05	6 of 70 documents move by USD 13,103.51.; US-NY-NYC USD 232,587.29 → 245,690.80; top mover CINV-US-2026-000016 +USD 2,500.40
C	Texas 0.0625 → 0.065 in Austin and Houston	7 of 70 documents move by USD 5,731.96.; Houston +4,963.95, Austin +1,065.01; three Stackworks supplier invoices move on accrued tax only
D	Rename a jurisdiction	No money moves; config-loads succeeds
E	Governance on a CR opened by Tam	Claude approve or merge → blocked SOD-02; Tam approve → blocked SOD-01; Maya approve → blocked SOD-03; Priya approve → approved; Priya merge → merged, <code>main</code> moved, ref log row <code>merge CR-0001: ...</code>
F	Stale approval	Commit again after Priya approved → approval shown stale; merge refused until re-check and re-approve
G	Agent proposes	"Raise the New York City rate to 5% from 1 October 2026" opens a change request as <code>claude</code> with the diff in B; checks run
H	Agent explains	The note's numbers all appear in the impact result; an injected wrong number is flagged
I	Audit chain	The verify action returns ok; editing a row in the database makes it fail
J	Hosted	Tests B, E and G pass at the public URL from a phone

13. Out of scope (all releases unless stated)

Inventory, manufacturing, payroll, fixed assets beyond a stub, full IFRS and management books, FX revaluation and translation (R3 candidate), a BPM engine, real filing or Peppol delivery, Pillar Two computation, production scale testing, and authentication before R3. Tax rates, thresholds and treaty positions are demonstration configuration marked for verification; the company Halide Labs and all its data are synthetic.

14. Risks and open questions

Risk	Mitigation
Forge's coding agent produces floating-point money or a replay that is off by rounding	The answer key and per-line half-up rule are acceptance criteria; the coding-agent instructions forbid floats for money
The accrual flag is ignored and Texas reports 4 documents instead of 7	Test C names the failure explicitly
Agent features cannot run at the hosted URL without an API key	Every agent action degrades to a clear message; the SOD refusal demo needs no model call
"Every incumbent ships governed agents" undercuts the pitch	Claim the mechanism (tested refusal, stored evidence, verified figures), never "AI inside the ERP"
The full vision is mistaken for the R1 deliverable	§7.1 is the contract for the day; everything else is tagged

Open questions: which tax engine partners to adapt first after the stub (Stripe Tax and Avalara are designed); pricing unit for hub mode (per entity or per jurisdiction); whether the merge queue should be R1 stretch or R2.

15. Glossary

Term	Plain meaning
ERP	Enterprise resource planning software: the system a company uses to keep its books and run buying, selling and paying.
Ledger / general ledger	The complete record of a company's accounting entries, organised by account.
Journal entry / journal line	One accounting transaction and its individual debit and credit lines. Debits must equal credits.
Book	A view of the ledger for a purpose (statutory, tax, management). Here the base book holds everything and an extension book holds only the differences.
Trial balance	The list of every account with its balance at a date; it nets to zero if the books balance.
Subledger	A detailed record for one area (payables, receivables, billing) that feeds the ledger. myERP has one ledger, so subledgers are just document types.

Term	Plain meaning
Document	A numbered business transaction (an invoice, an order, a payment) with lines and links to related documents.
Period close	The monthly process of finalising the books: reconciling, accruing, reviewing, locking.
Procure-to-pay / order-to-cash	The buying flow (order, receipt, supplier invoice, payment) and the selling flow (usage, invoice, receipt).
Sales tax, use tax, VAT, GST	Transaction taxes. Sales tax is charged by the seller (US). Use tax is what the buyer owes when the seller did not charge it. VAT and GST are the equivalents elsewhere, charged at each stage with credits for tax paid.
Reverse charge	A VAT rule where the buyer, not the seller, accounts for the tax on a cross-border business purchase.
Withholding	Tax a payer deducts from a payment and remits to the authority on the payee's behalf. It can happen to you (a customer short-pays) or by you (paying a foreign supplier).
Jurisdiction	A place that levies a tax (a state, a city, a country); rates stack (state + city + district).
Registration / nexus	Being registered to collect tax in a jurisdiction; nexus is the connection that obliges you to register.
Tax point date	The date that decides which rates and rules apply to a transaction.
Effective dating	Every rule has a "from" date and optionally a "to" date, so history stays correct when rules change.
Segregation of duties (SOD)	The principle that the person who prepares something cannot also approve it; here extended to AI agents.
Principal	Anyone or anything that acts in the system: a person, an AI agent, or the system itself.
Evidence run	A job that stores its inputs, outputs, control totals and a hash so an auditor can re-run it and get the same result.
Control total / tie-out	A sum used to prove two sets of records agree; a tie-out compares two such sums and reports the difference.
Hash / hash chain	A fingerprint of data; chaining each record's fingerprint to the previous one makes any edit detectable.
Commit, branch, merge, ref, diff	Version-control words: a commit is a saved version; a branch is a line of work; a merge brings it into the main line; a ref is a named pointer to a commit; a diff is the list of what changed.
Impact check	Replaying real documents through a proposed configuration to see what money it would move before approving it.
SOX	The US law requiring public companies to prove their financial controls work.
MCP	Model Context Protocol, a standard way to expose tools to AI agents.
dbt	A tool that builds a data warehouse from SQL models with tests.
Grounding check	A deterministic check that every number an AI states came from a tool result it actually saw.

Appendix A — Seed data (repository folder /data)

File	Rows	Content
entities.csv	4	Halide Labs US, Ireland, Japan, Singapore
accounts.csv	51	Chart of accounts
principals.csv	7	People, Claude, system
parties.csv	35	Customers and suppliers
documents.csv	70	Tax-determined customer and supplier invoices with totals
tax_lines.csv	608	Tax lines per document line and jurisdiction
journal_entries.csv	133	Accounting documents
journal_lines.csv	1,147	The universal journal (balanced per currency)
registrations.csv	24	Entity tax registrations
rates.yml	15 / 27	Jurisdictions and rate components, effective-dated
process_definitions.yml	—	SOD rules and the config_change process
answer_key.json	4 scenarios	Expected impact results (§12 A–D)

Business date: 2026-09-16. Halide Labs is fictional; amounts are integer minor units.

Appendix B — The reference implementation, and what R1 reuses

A hand-built reference of this product exists (Python, PostgreSQL, React; 142 tests; built by the author over September 2026 with Claude Code). It implements FR-LED through FR-EVD as R2 describes them, the agent layer of §6.2 through the close agent and voice, and change control with the impact check. **The Forge build is a new codebase and does not copy from it.** It reuses two things: the seed data in Appendix A (exported from the reference database) and the answer key (computed by the reference by replaying the same data). This is disclosed in the demo and in the write-up.

Appendix C — The governed tool registry in the reference (for R2's Ask capability)

Read tools: get_close_overview, query_metric, drilldown_metric, get_tax_summary, search_journal_lines, explain_journal_line, get_trial_balance, search_documents, get_document, get_withholding_overview, get_nexus_exposure, get_tax_registrations, get_receivables_aging, list_evidence_runs, get_evidence_run, get_integrity_status. Evidence tools: run_indirect_tax_recon, run_ledger_integrity_check. Propose tools: propose_journal_entry, propose_task. Control tool: submit_proposal_for_approval (runs the real approval step as the agent and is refused by SOD-02). Twenty-one tools; the same registry serves the in-app agent, the voice surface and the MCP server. R1's Ask stretch uses the first seven.